Replication of Flakify: A Black-Box, Language Model-Based Predictor for Flaky Tests

Namu Go
ng2349@nyu.edu
New York University Abu Dhabi
Abu Dhabi, Abu Dhabi, United Arab Emirates

Nelson Mbigili
nfm8340@nyu.edu
New York University Abu Dhabi
Abu Dhabi, Abu Dhabi, United Arab Emirates

Abstract

This paper is a report on the replication study of *Flakify* (Flaky Test Classify), a black-box machine learning solution for detecting flaky tests in software development. The original paper used two different datasets (FlakeFlagger and IDoFT) as well as two different validation schemes (cross-validation and per-project validation) for evaluation, and the authors report that they obtained F1 scores ranging between 73% and 98% depending on the dataset and the validation method. We evaluated these findings by first manually analyzing flakiness in 4 sample tests from the FlakeFlagger dataset. Second, reproducing the training process of the Flakify model and third, running it on selected tests from a different new dataset, FlakyCat. We observe that the labels in the FlakeFlagger dataset are accurate with Flaky tests actually justifiable flaky and non flaky alike. Additionally the the retrained model largely shows a similar performance as described in the original paper, with the model trained on IDoFT, correctly marking all 20 tests as flaky from the FlakyCat dataset and the model trained on FlakeFlagger marking only 5 out of 20 as Flaky.

ACM Reference Format:
Namu Go and Nelson Mbigili. 2026. Replication of Flakify: A Black-Box, Language Model-Based Predictor for Flaky Tests. In *NYU Abu Dhabi Software Analytics Replication Series.* ACM, New York, NY, USA, 3 pages.

1 Introduction

In software testing, *flaky test* refers to a test that shows non - deterministic behavior. A test is supposed to catch errors or bugs in the logic or syntax of the source code and therefore has to give the same result for the same source code, but a flaky test passes and fails across different executions even though the source code stays the same. [3] Recent research has shown the potential for effectiveness and scalability of a machine learning (ML) approach to detecting flaky tests without having to rerun the test cases [2], but such solutions still suffer from inaccessibility of production codes and low accuracy.

Addressing the limitations of these ML approaches for flaky test detection, Fatima et al. [4] presented Flakify, a black-box solution that does not require access to production code, rerunning test cases, or knowledge of pre-defined features. Specifically, they have used CodeBERT [5], a pretrained language model, to make predictions based on a given source code. The paper demonstrated that Flakify exhibited a major improvement on accuracy and generalizability.

Our work aims to verify the findings of Fatima et al. [4], specifically by replicating the results of Research Question 1 - *How accurately can Flakify predict flaky test cases?* To further test the generalizability of Flakify, we selected 20 tests from a dataset not used in the original paper called Flakycat [1] and observed the performance of Flakify on them.

All artifacts related to this replication study are available online.[1] Additionally, due to storage limitations, the weights for our trained models can be found in a different repository. [2]

2 Replication Study Details

2.1 Summary of Original Study

The flaky test predictor Flakify as introduced in the original study [4] emphasized the solution being a purely black-box approach. To achieve this, the authors fine-tuned CodeBERT, a transformer-based language model trained on programming languages, to classify test methods as either flaky or non-flaky.

Because CodeBERT has a strict 512-token input limit, the authors applied a specialized pre-processing step to the test codes. They tokenized inputs using a WordPiece tokenizer and prioritized retaining aspects associated with common test smells (e.g., Async Wait, Resource Optimism). The study utilized two primary datasets: FlakeFlagger [2] and IDoFT [3]. To evaluate their model (RQ1), they employed 10-fold stratified cross-validation alongside random oversampling to handle the severe class imbalance inherent to flaky test datasets. They reported highly effective results, achieving an F1-score of 79% on the FlakeFlagger dataset and 98% on the IDoFT dataset.

2.2 Study Design

Our replication study is designed in three distinct phases to fulfill the defined scope of replication and extension:

- We began by sampling four test cases from the FlakeFlagger dataset, two tests labeled as flaky and two labeled as non-flaky, and manually inspected the source code of these tests to determine whether we agreed with their provided labels.
- We performed a complete retraining of the model from scratch, using both the FlakeFlagger and IDoFT datasets. The performance metrics derived from our retraining were then directly compared to the benchmark results reported by Fatima et al. [4].
- To evaluate the model's generalizability on unseen data, we passed a sample of 20 test files from the FlakyCat [1] dataset through our retrained Flakify model for classification. We recorded the exact number of these tests that the model successfully flagged as flaky, and inspected the results.

[1] https://github.com/Nelsonmbigili/Replication_FlakyTestsPredictor
[2] https://huggingface.co/Nelsonmbigili/Replication_flakify_weights/tree/main
[3] https://github.com/TestingResearchIllinois/idoft?tab=readme-ov-file

3 Results

3.1 Manual Inspection of Labels

We analyzed two tests labeled as deterministic. The first, assertExecuteWhenFetchDataIsNotEmptyForStreamingProcessAndMultipleShardingItems (from elastic-job-lite), involved complex execution logic but was still deterministic because all external dependencies were stubbed using the Mockito framework. The second test, randomValue (from spring-boot), was correctly labeled as non-flaky because its assertion checked for notNullValue() rather than asserting against the randomly generated value itself.

We reviewed two tests labeled as non-deterministic and identified clear test smells in both. The first, oldSpringModulesAreNotTransitiveDependencies (from spring-boot), completely lacked assertions. It relied entirely on the runBuildForTask function executing without throwing an error. The second test, cannotProcessInvalidCss (from wro4j), contained a logical contradiction: it was annotated to expect a WroRuntimeException, yet it attempted to assert that the method returned an empty string. This contradictory design creates multiple, non-deterministic execution paths.

3.2 Flakify Retraining

Due to severe computational constraints and unforeseen runtime interruptions during the extended fine-tuning process, we were unable to complete the entire retraining pipeline for the FlakeFlagger dataset. Consequently, our reproduction of RQ1 focuses exclusively on the IDoFT dataset evaluated under the 10-fold cross-validation scheme.

In the original study, Fatima et al. [4] reported an exceptionally high performance on the IDoFT dataset via cross-validation, achieving an F1-score of 98% (Precision: 99%, Recall: 96%). Our independent retraining of the CodeBERT model on the IDoFT dataset yielded strong but lower results compared to the original benchmarks. Specifically, our replicated model achieved a Precision of 91.01%, a Recall of 82.68%, and an F1-score of 86.64%, alongside an overall accuracy of 78.43%.

Table 1: Results of Flakify (cross-validation, IDoFT)

Approach	Dataset	Precision	Recall	F1-Score
Flakify	IDoFT dataset	99%	96%	98%

While our F1-score sits approximately 11 percentage points below the original 98%, the model still demonstrated a highly capable baseline for predicting flaky tests purely from source code. The variance between our metrics and those of the original authors is common in deep learning replications and may be attributed to slight differences in batch sizing, random seed initialization, or the impacts of our hardware constraints during the fine-tuning process. Ultimately, these results support the original authors' claim that language models can reliably extract flakiness indicators from test code.

3.3 FlakyCat Classification

To evaluate the model's cross-project generalizability, we applied our retrained Flakify model to a sample of 20 known flaky test files from the external FlakyCat dataset. Because all 20 of these selected tests are definitively flaky, the higher the number of tests flagged as flaky, the better the model performs.

To gain deeper insights, we evaluated these 20 tests twice, utilizing two different sets of pretrained weights obtained from our Phase 2 replication attempts: one from the FlakeFlagger training run and one from the IDoFT training run.

- FlakeFlagger-Trained Model: When using the weights from the FlakeFlagger training run, the model correctly flagged only 5 of the 20 tests (25%) as flaky.
- IDoFT-Trained Model: When using the weights from the IDoFT training run, the model correctly flagged 20 out of 20 tests (100%) as flaky.

The stark contrast in performance between the two models is highly consistent with the circumstances of our Phase 2 training phase. As noted previously, the training process for the FlakeFlagger dataset was prematurely terminated due to runtime constraints. The poor predictive performance of this model on the FlakyCat dataset is seen to be a direct consequence of this incomplete fine-tuning.

However, the IDoFT-trained model, which successfully completed its entire fine-tuning pipeline, demonstrated exceptional generalizability by correctly identifying 100% of the unseen flaky tests. Furthermore, this discrepancy aligns with the findings of the original study by Fatima et al. [4], which reported that the model achieved significantly higher performance metrics when trained on the IDoFT dataset (98% F1-score) compared to the FlakeFlagger dataset (79% F1-score).

4 Replication Conclusions and Insights

In conlusion, through this replication work we make the following contributions:

- Manually analyze Flaky status labels in the FlakeFlagger dataset for 4 randomly selected tests.
- Re-run the training of the clasification model using FlakeFlagger dataset and the IDoFT dataset.
- Run *Flakify* on a sample of 20 tests from new unseen dataset (FlakyCat).

In our manual analysis of the 4 flaky tests we find that the flaky status labels in the FlakeFlagger dataset are accurate as the tests marked flaky are reasonably flaky and those that are marked as non flaky are justifiably so. Additionally, we retrain the model and on running *Flakify* on the sample tests from FlakyCat we find that using FlakeFlagger trained weights, *Flakify* identifies of 5 of 20 as flaky while using IDoFT correctly identifies all 20 tests as Flaky. In general, the successful reproduction of results on the original work confirms the model's architectural soundness, while the high capture rate on the new FlakyCat data demonstrates that the model has learned generalized, project-agnostic flakiness patterns rather than just memorizing project-specific artifacts, especially using the IDoFT trained weights. While the provided codebase was well-structured and easy to follow, the training process was extremely time-consuming and ideally not possible with our local machines, making access to a High Performance Computer necessary.

5 Statement of Contributions

Both authors of this study worked together in person on source code analysis and training configuration adjustment.

Nelson Mbigili primarily worked on manually analyzing the flaky status labels for replication task 1, sampling tests from FlakyCat and writing the modified script to run*Flakify* on new dataset.

Namu Go primarily worked on running and troubleshooting the training script for Phase 2 and analyzing the results. He wrote all sections of this report except 4. Conclusions.

References

[1] Amal Akli, Guillaume Haben, Sarra Habchi, Mike Papadakis, and Yves Le Traon. 2023. FlakyCat: Predicting Flaky Tests Categories using Few-Shot Learning. In *2023 IEEE/ACM International Conference on Automation of Software Test (AST)*. 140–151. doi:10.1109/AST58925.2023.00018

[2] Abdulrahman Alshammari, Christopher Morris, Michael Hilton, and Jonathan Bell. 2021. FlakeFlagger: Predicting Flakiness Without Rerunning Tests. In *Proceedings of the 43rd International Conference on Software Engineering* (Madrid, Spain) *(ICSE '21)*. IEEE Press, 1572–1584. doi:10.1109/ICSE43902.2021.00140

[3] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding flaky tests: the developer's perspective. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estonia) *(ESEC/FSE 2019)*. Association for Computing Machinery, Abu Dhabi, AD, UAE, 830–840. doi:10.1145/3338906.3338945

[4] Sakina Fatima, Taher A. Ghaleb, and Lionel Briand. 2023. Flakify: A Black-Box, Language Model-Based Predictor for Flaky Tests. *IEEE Transactions on Software Engineering* 49, 4 (April 2023), 1912–1927. doi:10.1109/tse.2022.3201209

[5] Monique Louise Monteiro, George Gomes Cabral, and Adriano Lorena de Oliveira. 2025. Pre-trained Code Language Models fornbsp;Just-in-Time Software Defect Prediction: An Empirical Study. In *Intelligent Systems: 35th Brazilian Conference, BRACIS 2025, Fortaleza-CE, Brazil, September 29 – October 2, 2025, Proceedings, Part II* (Fortaleza-CE, Brazil). Springer-Verlag, Berlin, Heidelberg, 347–361. doi:10.1007/978-3-032-15984-7_24